Guía del proyecto

Detección de RoboMasters y conducción autónoma

PIDS 2026 – Grupo 2 – UPM

Documento de estudio para la presentación del lunes

Estado del repositorio a 13/11/2026 (rama `main` y `javier-saguar` al día)

Este documento no repite lo básico (qué es una CNN, qué es el IoU…). Recoge qué hay en el proyecto, *por qué* se decidió así y los detalles que más suelen preguntar. Las cifras salen de la memoria de la entrega 3, de los documentos de `docs/` y del código de `ENTREGA 2/WEEK3`.

Índice

1. El proyecto en una página **2**

2. Bloque 1: los datos **3**

2.1. Recolección . . . 3
2.2. Extracción: por qué `INTERVAL = 24` . . . 3
2.3. Anotación: criterios y consistencia . . . 3
2.4. EDA: qué se vio y qué se decidió con ello . . . 3
2.5. Depuración: 808 → 796 . . . 4
2.6. Aumento de datos . . . 4

3. Bloque 2: el modelo **5**

3.1. El sistema de entrenamiento . . . 5
3.2. Las siete arquitecturas . . . 5
3.3. Hiperparámetros y su porqué . . . 6
3.4. Cronología de los experimentos . . . 7
3.5. Resultados en test . . . 7
3.6. Limitaciones honestas (úsalas, suman puntos) . . . 8

4. Bloque 3: el robot **8**

4.1. Arquitectura del software . . . 8
4.2. El circuito . . . 9
4.3. Detección, puntería y disparo: lo que hay que entender . . . 10
4.4. Los problemas reales y cómo se resolvieron . . . 11
4.5. El robot camuflado (y la lección sobre aliados) . . . 11
4.6. Resultados de la demo oficial (12/11, en clase) . . . 12
4.7. Seguridad (muy preguntable) . . . 13
4.8. Mejoras implementadas pero desactivadas . . . 13

5. La interfaz del mando **14**

6. Material de apoyo para la presentación **14**

7. Mapa de decisiones: qué salió bien, qué falló y por qué **16**

7.1. Decisiones que salieron bien . . . 16
7.2. Lo que falló (o se descartó) y la lección . . . 16

8. Preguntas probables y cómo responderlas **19**

9. Cifras para tener en la cabeza **20**

1. El proyecto en una página

Qué hace. Un RoboMaster EP recorre un circuito, se para en puntos elegidos, busca con la cámara de su torreta a otros RoboMasters, los detecta con una red neuronal entrenada por nosotros, apunta y les dispara 20 ráfagas de infrarrojos (el enunciado exige 12 impactos). Todo el proceso (datos, modelo, robot) lo hemos hecho nosotros.

Bloque	Qué se hizo	Resultado
1. Datos	10 vídeos propios → 808 fotogramas anotados a mano → 796 tras depurar → 3184 con aumento	Dataset con 1420 robots (375 *difficult*), IoU medio entre anotadores 0,81
2. Modelo	7 arquitecturas comparadas, 3 tasas de aprendizaje, test usado una sola vez por modelo	**Faster R-CNN v2**: AP@0,5 = 0,9523 en test (0,6921 en *difficult*), umbral 0,95
3. Robot	Demo autónoma en el circuito + interfaz web de mando con detector en GPU	Demo oficial del 12/11: los 4 rivales eliminados, 80 disparos registrados

Idea clave

El hilo conductor de toda la presentación es la **justificación de las decisiones**, que es lo que más valora la asignatura: "una demostración imperfecta bien explicada puntúa más que una perfecta sin analizar". Cada decisión de este documento tiene su porqué; aprende el porqué, no solo la cifra.

Lo que hay de nuevo (revisado hoy)

- **Javi no ha subido nada nuevo**: su último trabajo es la PR #43 (fusionada) y el commit del 12/11 con el Bloque 3 en GitHub Pages.
- **Documentos desfasados:** `ESTADO.md` (04/11) y `circuito_paso_a_paso.md` (08/11, aún habla de R1 y R2 como "supuestos"). La fuente de verdad es la **memoria de la entrega 3**, actualizada el 12/11.
- La memoria se entrega el 15/11 y tiene 32 páginas de cuerpo frente a un límite de 30: conviene comprobarlo.

2. Bloque 1: los datos

2.1. Recolección

Diez vídeos de 1280×720 a 30 fps con la cámara del robot (modo *Solo*), 643,8 s en total y 19 508 fotogramas, en seis escenarios: vestíbulo de granito pulido, vestíbulo de suelo de color, aula con sillas, aula blanca, pasillo largo y exterior. Variabilidad buscada: interior/exterior, luz natural/artificial/mixta, oclusiones por sillas y piernas, de 1 a 3 robots, cámara en movimiento.

Limitaciones que conviene reconocer: los dos exteriores (V04 y V09) son del mismo sitio y no hay secuencias del circuito real ni con muy poca luz. Es el tipo de "análisis honesto" que premia el enunciado.

2.2. Extracción: por qué `INTERVAL = 24`

Se guarda un fotograma de cada 24 (≈0,8 s). Con 20 saldrían 972 imágenes y con 30, 645; con 24 salen **808**, justo lo que pide el enunciado (∼800) sin guardar fotogramas casi idénticos.

2.3. Anotación: criterios y consistencia

Herramienta `labelImg`, formato PascalVOC, una sola clase (`robot`). Los criterios están en `docs/criterios_anotacion.md`. Los que más se preguntan:

Regla	Criterio	Por qué
Antenas	La caja **no** las incluye (chasis, ruedas, gimbal y cañón sí)	Solo se ven de cerca: con ellas el tamaño y el centro de la caja dependerían de la distancia
Nuestro cañón	No se anota	Aparece en todas las imágenes; anotarlo enseñaría a detectarse a sí mismo
Tapados / cortados	Se anotan si se ve al menos un 30 % y la caja cubre solo lo visible	Es el mismo 30 % que usa el aumento (`min_visibility`)
difficult	<30 px de ancho, menos de la mitad visible o muy movido	Permite medir el modelo aparte en los casos difíciles
Imágenes sin robots	Se guardan con `.xml` vacío	Sin `.xml` el entrenamiento las ignora y se pierden como negativos
Duda	"¿Si viera solo este fotograma, diría que hay un robot?"	La red ve las imágenes de una en una, sin vídeo

Consistencia. Los cinco anotamos las mismas 20 imágenes (elegidas por difíciles): IoU medio entre parejas **0,81**. Donde más discrepamos: el **borde superior** (mediana de 7 px frente a 4–5 px en los demás), es decir, gimbal y antenas. Después, cada lote lo revisó otra persona (revisión cruzada). El reparto de lotes: Javier 01–02, Alejandro 03–04, Mónica 05–06, Pedro 07–08, Daniel 09–10.

Pendiente reconocido (T01): cerrar en grupo el criterio del borde superior.

2.4. EDA: qué se vio y qué se decidió con ello

808 imágenes, 1420 robots (1,76 por imagen), 375 *difficult* (26 %), 41 imágenes sin robots (5 %). Lo interesante no es el inventario, sino que **cada análisis dirigió una decisión posterior**:

Análisis	Qué se vio	Decisión
Tamaño de cajas	Ninguna caja <30 px de ancho	Anclas por defecto sirven; el aumento añade robots más pequeños (alejamiento)
Forma y anclas	Cajas casi cuadradas (aspecto mediano 1,11); las anclas cubren el 99,6 % con IoU $\geq$ 0,5	Mantener anclas; el aumento **no puede deformar** los robots
Posición de centros	Franja horizontal a media altura	Volteo horizontal sí, vertical no
Solapamiento	Ninguna pareja solapa más de 0,26	El NMS por defecto (0,5) no borra robots reales
Parecido entre imágenes	El vecino más parecido es del mismo vídeo en el 74 % de casos	No mezclar fotogramas de un vídeo entre entrenamiento y evaluación
Nitidez y duplicados	52 poco nítidas, 11 casi duplicadas	Revisarlas en la depuración

2.5. Depuración: 808 → 796

Se revisaron a ojo 69 imágenes. Se excluyeron **12**: 1 con un robot visible pero sin caja (solo una estela: etiqueta contradictoria) y 11 casi duplicadas (se conserva la primera del tramo). *No* se borra ningún original: la exclusión se aplica al generar los conjuntos (`docs/depuracion_excluir.csv`). Se conservan 41 imágenes movidas con sus robots como *difficult* (el robot se moverá en la demo) y los negativos difíciles (piernas, nuestro propio robot).

Pregunta probable: ¿Por qué quedáis por debajo de 800?

Se prefirió un dataset sin etiquetas contradictorias a llegar a 800 justos; quedan 796. Estaba prevista una grabación complementaria si hubiera hecho falta.

2.6. Aumento de datos

Reparto de conjuntos (T18): se entrena con *nuestros* 796 originales; la validación (672 imágenes, 1561 robots, 255 *difficult*) y el test (718 imágenes, 1721 robots, 270 *difficult*) son los conjuntos de otros grupos que da la asignatura. Así no puede haber fuga entre conjuntos y se mide en escenas no vistas. Esas imágenes venían sin *difficult*: los marcamos nosotros con nuestro criterio, sin tocar las cajas.

Se generan 3 copias aumentadas por imagen: **3184 imágenes de entrenamiento** (796 + 2388). Transformaciones usadas: volteo horizontal, escala 0,6–1,25 sin deformar, giro ±6°, brillo/contraste/gamma, tono y saturación suaves, desenfoque de movimiento, JPEG/ruido, y una **oclusión sintética propia** (barras tipo patas de silla que tapan como máximo el 35 % de un robot). Descartadas, con motivo: volteo vertical y giros de 90° (el suelo nunca está arriba), el recorte 640×480 del notebook original (ensancha los robots un 33 %), desenfoque uniforme, intercambio de canales, perspectiva (en albumentations 1.4.18 desplaza mal las cajas, detectado con una prueba automática) y relleno por reflejo (duplica robots sin caja).

Dato defendible: el aumento propio pierde el 0,7 % de los robots; el del notebook original, simulado, perdería el 14,1 % y los deformaría (aspecto 1,18 frente a 1,10).

El perfil final de entrenamiento es `robusto_v1`: suma sombras locales y lluvia (solo para los vídeos de exterior), oclusiones con textura y refuerzo de copias con *difficult*.



Figura 1: Ejemplo de aumento de datos.

3. Bloque 2: el modelo

3.1. El sistema de entrenamiento

Código en `train.ipynb` y `test.ipynb` (JupyterHub del laboratorio, GPU RTX 3060 de 12 GB, PyTorch 2.4.1). El esquema de la memoria es: datos → carga en lotes → detector → función de pérdida (RPN + clase + posición) → optimizador SGD → evaluación en validación cada época → selección del mejor *checkpoint* → test (una vez).

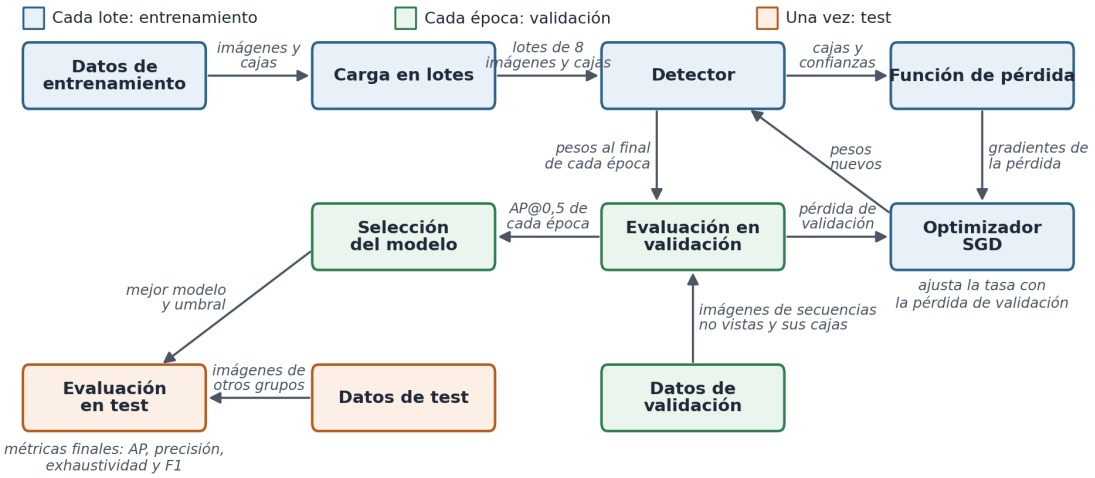


Figura 2: Esquema de bloques del entrenamiento y la evaluación.

3.2. Las siete arquitecturas

Todas con **transfer learning** desde COCO, sustituyendo la cabeza de clasificación por una de dos clases (fondo/robot). Con 796 imágenes originales sería inviable entrenar desde cero.

Modelo	Tipo	Backbone	Para qué sirve
Faster R-CNN v1 / v2	Dos etapas (RPN + clasificación por región)	ResNet-50 + FPN	v2 tiene mejoras de cabezas y entrenamiento
Faster R-CNN MobileNet (v3)	Dos etapas, ligera	MobileNetV3-L + FPN	Opción rápida (16 ms)
RetinaNet v1 / v2	Una etapa, pérdida focal	ResNet-50 + FPN	Compensa el desequilibrio de anclas de fondo
SSD300 (ssd_v1)	Una etapa, entrada 300×300	VGG16	Rápida, pierde robots lejanos
SSDlite320 (ssd_v2)	Una etapa, ligera	MobileNetV3-L	La peor (AP 0,64)

Detalles que suelen preguntar:

- **FPN:** recorre los mapas de características de arriba abajo, amplía cada mapa profundo al tamaño del anterior y lo suma tras una convolución 1×1. Resultado: pirámide de mapas con información semántica en todos los niveles; los robots lejanos se buscan en los mapas grandes y los cercanos en los pequeños.
- **Una etapa frente a dos:** dos etapas localizan mejor (la segunda trabaja con pocas regiones ya filtradas), una etapa es más simple y rápida pero debe separar pocos objetos de decenas de miles de anclas de fondo.
- **SSD reduce a 300×300 / 320×320**, por eso los robots lejanos quedan minúsculos y los *difficult* caen (AP 0,43 y 0,27).

3.3. Hiperparámetros y su porqué

- **Lote 8.** Es el mayor que cabe en 12 GB; con 16 y 32 no cabe. Pruebas cortas: lotes de 1 y 2 suben antes pero bajan en la tercera época; con 8 sube más regular. Faster R-CNN v2 y el experimento a 1024 px no caben con 8: se usan **microlotes de 4 acumulando gradientes** (lote efectivo 8). Se prueban 8, 4 y 2 (los que dividen a 8) y no 6, para que todos los pasos tengan 8 imágenes y sean comparables.
- **SGD, momentum 0,9, weight decay** $5 \cdot 10^{-4}$**:** valores del artículo de Faster R-CNN y del tutorial de torchvision; no se tocaron porque solo se ajustó la tasa, el hiperparámetro al que el entrenamiento es más sensible. No se probaron otros optimizadores, así que no se puede decir que SGD sea mejor.
- **Tasa de aprendizaje.** 10^{-4} (lenta: mejor época 29), 10^{-3} (mejor época ≈5), $2 \cdot 10^{-3}$ (el máximo llega en la época 1–2 y luego baja más de 1 pp), 10^{-5} (casi no aprende), 10^{-1} (NaN). Entre 10^{-4} y 10^{-2} la tasa cambia sobre todo la velocidad, no el resultado.
- **Programador:** multiplica la tasa por 0,1 tras 5 épocas sin mejorar la pérdida de validación. La memoria reconoce que la paciencia de 5 resultó larga (el estancamiento ocurre en 2–3 épocas) y propone una estrategia más sistemática (prueba de rango de la tasa).
- **Épocas:** 30 al inicio, 12 después (15 con lr $5 \cdot 10^{-4}$). En 6 de 7 modelos el mejor *checkpoint* está entre las épocas 3 y 7: hay **sobreajuste temprano**, no falta de épocas. Se comprobó con SSD300 a 40 épocas: el AP en validación no se mueve.

3.4. Cronología de los experimentos

1. **Aumento sí/no** (Faster R-CNN v1, lr 10^{-4}): AP@0,5 en validación 0,923 → 0,942; en *difficult* 0,552 → 0,634 (+8 pp). Pierde un 28 % menos de robots (FN 115 → 83) sin añadir falsos positivos. Se toma como base.
2. **Tasa 10^{-3}:** AP@0,5 0,9433, AP 0,50:0,95 sube de 0,608 a 0,630, FP 210 → 190. Mejor época 5, no la última.
3. **Resolución 1024:** AP@0,5 0,9465 y AP pequeños 0,12 → 0,19, pero 134 FP más, AP 0,50:0,95 menor y latencia +62 % (75 → 122 ms). **Se descarta.** Moraleja: no mejora de forma uniforme.
4. **Cadena de 13 entrenamientos** (siete arquitecturas + variantes de tasa de las tres mejores) con el perfil `robusto_v1`, automatizada con `herramientas/cadena_entrenamientos.py` (preflight, reanudable, microlotes automáticos).
5. **Test, una sola vez por modelo**, con el umbral fijado antes en validación (mejor F1).

La regla de selección se escribió antes de abrir el test (`docs/seleccion_modelo_final.md`, 24/10): mayor AP@0,5 en validación; si la diferencia es <0,5 pp (una sola semilla) se considera empate y se prefiere el más barato; el test no se usa para elegir nada.

3.5. Resultados en test

Modelo	AP val.	AP test	Diff. test	Umbral	F1	FP/FN	ms
Faster R-CNN v2	0,9513	**0,9523**	**0,6921**	0,95	0,9389	94/115	96
RetinaNet v2	0,9447	0,9457	0,6313	0,65	0,9233	98/161	67
Faster R-CNN v1	0,9426	0,9355	0,6057	0,90	0,9237	134/129	75
RetinaNet v1	0,9396	0,9362	0,6273	0,65	0,9218	133/136	72
Faster R-CNN MobileNet	0,9281	0,9240	0,5744	0,55	0,9142	147/148	16
SSD300	0,9008	0,8797	0,4316	0,25	0,8648	135/307	12
SSDlite320	0,6823	0,6412	0,2742	0,20	0,6571	384/691	4

- **Test confirma validación:** en el modelo final el AP es prácticamente el mismo (0,9513 → 0,9523). Los ResNet-50 apenas cambian; las SSD pierden más.
- **Modelo final:** Faster R-CNN v2, lr 10^{-3}, época 3, umbral 0,95. Con él: 1606 TP, 94 FP, 115 FN; precisión 0,9447, exhaustividad 0,9332. AP@0,75 = 0,6005 y AP 0,50:0,95 = 0,5724: *encuentra casi todos, pero las cajas no se ajustan tanto.*
- **Dónde falla:** 74 de los 115 FN (64 %) son *difficult*, que son solo el 16 % de los robots. Subir el umbral de 0,5 a 0,95 baja los FP de 226 a 94 pero baja la exhaustividad en *difficult* de 0,84 a 0,73.
- **Alternativa rápida:** Faster R-CNN MobileNet, 6 veces más rápido (16 ms), AP 0,924.

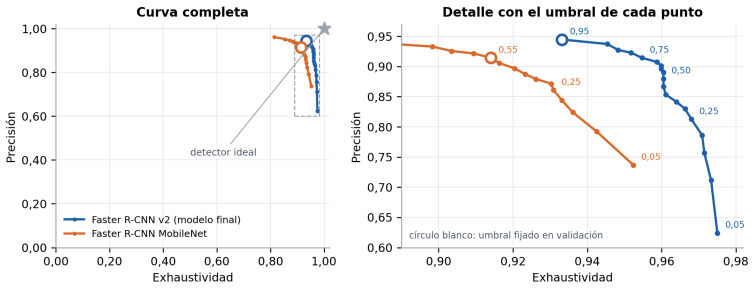


Figura 3: Curva precisión–exhaustividad en test del modelo final.

3.6. Limitaciones honestas (úsalas, suman puntos)

- **Una sola semilla:** diferencias <0,5 pp no son concluyentes.
- **Etiquetas de validación/test de otros grupos**: no siguen del todo nuestro criterio (antenas, robots cortados) y algunas están incompletas. Caso confirmado: `PIDS2024_GR02_0001/frame_0444.jpg` tiene **4 robots visibles pero 1 sola caja** en el XML, así que detectar los otros 3 cuenta como FP. No se corrigieron las etiquetas para conservar la comparación, y no se sabe cuántos FP se deben a esto.
- **Secuencias difíciles:** en test, `PIDS2025_GR02_0009` pierde 40 de 204 robots y `PIDS2025_GR04_0000` pierde 10 de 21.
- **Robots lejanos:** AP de objetos pequeños de 0,12–0,19; no hay cajas originales de menos de 30 px.
- La mejor época se eligió con validación: sus cifras son algo optimistas (por eso existe el test).

Pregunta probable: ¿Por qué umbral 0,95, tan alto?

Es el de mejor F1 en *validación* (0,9361), fijado antes de ver el test. En test, el mejor posible habría sido 0,9413 con umbral 0,90: solo 0,24 pp más, así que el umbral generaliza. Además, en la demo **es preferible no disparar a algo que no es un robot**: un falso positivo es peor que perder un objetivo difícil.

Pregunta probable: ¿Por qué el modelo final y no el MobileNet, que es 6 veces más rápido?

Por mayor AP y mejor F1. La latencia importa menos de lo que parece: en el robot el cuello de botella no era el modelo sino el diseño de la demo (ver Bloque 3). Además se resolvió llevando la inferencia a la GPU (0,12 s por imagen). MobileNet queda como plan B.

4. Bloque 3: el robot

4.1. Arquitectura del software

Todo en `ENTREGA 2/WEEK3`. La separación de responsabilidades es la clave para explicarlo:

`circuito.py`	Dice **qué** hacer: modelo, umbral, paradas, rivales, velocidades. Es lo único que se ajusta en el laboratorio.
`recorrido.py`	Convierte el plano (metros) en movimientos del chasis.
`demo.py`	Decide **cuándo** hacer cada cosa (el director: barrido, puntería, ataque en marcha, baile).
`demo_utils.py`	Sabe **cómo** se hace cada acción con el robot: cámara, detector, puntería, avance recto, LED, sonido.
`demo_gpu.py`	La misma demo, cambiando el detector por uno en GPU (proceso aparte).
`simulacion.py`	Robot simulado: la demo completa se prueba sin RoboMaster, con imágenes de test.
`rival/`	Kit para mover el rival 2 desde otro portátil.
`mando/`	Interfaz web de control manual.

Restricción de partida: el SDK del RoboMaster solo funciona con **Python 3.8**, y el PyTorch de 3.8 no admite la RTX 5070 del portátil. Resultado: en CPU cada imagen tarda 2,2–2,4 s. *Esa latencia condiciona todo el bloque*: el robot no puede mirar y moverse a la vez.

Solución: un proceso aparte con Python 3.11 y CUDA 12.8 carga el mismo modelo; la demo le manda cada imagen en JPEG por HTTP local (`127.0.0.1:8767`). De 1,3 s a 0,12 s por imagen (en simulación), con las mismas detecciones. Si la GPU falla, esa imagen va a CPU; tras 3 fallos seguidos, el resto de la demo en CPU. La versión original se conserva.

Idea clave

Es una buena respuesta a "¿qué problema técnico fue el más gordo?": no fue la red, fue integrar un SDK atado a Python 3.8 con una GPU moderna. Y la solución fue arquitectónica (proceso separado + respaldo), no un parche.

4.2. El circuito

Plano en metros con origen en la salida; x adelante, y a la derecha. Las ruedas **mecanum** permiten moverse de lado y hacia atrás sin girar el chasis. Todos los tramos se midieron en el laboratorio.

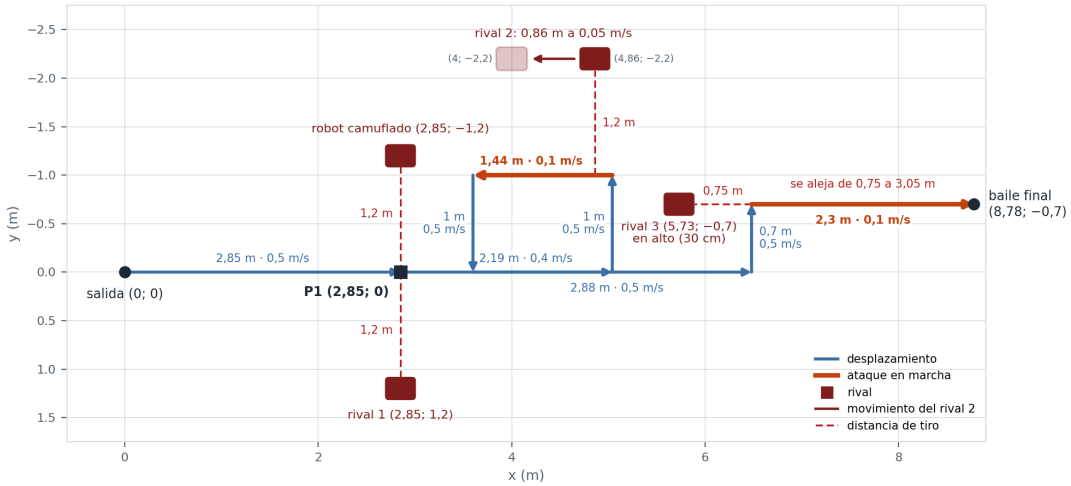


Figura 4: Recorrido de la demo (azul: desplazamientos; naranja: ataques en marcha; granate: rivales; rojo discontinuo: distancia de tiro).

Tramo	Movimiento	Velocidad	Acción
P1	2,85 m adelante	0,5 m/s	Suena un Fórmula 1 ajustado a la recta (6,4 s)
P1	Parada	–	Torreta a 90°: rival 1; a −90°: robot camuflado
R1	2,19 m adelante, 1 m izq.	0,4 / 0,5	–
R1	1,44 m **atrás**	0,1 m/s	Torreta a la izquierda: **rival 2, que se mueve** 0,86 m a 0,05 m/s
R1	1 m derecha	0,5 m/s	–
R2	2,88 m adelante, 0,7 m izq.	0,5 m/s	–
R2	2,3 m adelante	0,1 m/s	Torreta a 180°: **rival 3, escondido** (se busca con pitch 10°)
Final	Baile	–	Super Estrella, torreta, chasis y LED

Por qué se retrocede a 0,1 m/s en los ataques en marcha: el tiempo se reparte así: girar la torreta 90° (3 s) + ver al rival (2–4 s) + aviso de voz (1,4 s) + ráfaga (6,2 s). A 0,1 m/s los 1,44 m duran ≈14,5 s, suficiente para todo. La velocidad sale de *sumar tiempos*, no de ajustar a ojo.

4.3. Detección, puntería y disparo: lo que hay que entender

1. **Píxeles a grados** (`auto_aim`): $\alpha = \arctan(\text{desvío}/f)$ con $f = \text{ancho}/(2\tan 60°)$, escalado por 90/120 porque el campo de visión medido es ≈90° y no 120°. Es la función que da la plantilla de la asignatura y la hemos extendido a vertical (`auto_pitch`).
2. **Distancia** sin sensor: $d = \text{ALTURA_ROBOT}/(2\tan(\text{alto angular}/2))$. Aproximada, pero separa 1,3 m de 2,6 m; sirve para descartar robots del fondo.
3. **Puntería por pasos:** se gira hacia el ángulo estimado, se toma otra imagen, se mide el error y se corrige hasta 4 veces; se dispara cuando el error es <3° o <30 % del ancho de la caja. Solo se usan giros *relativos*, así que un error acumulado en el ángulo absoluto no importa. Si al confirmar no ve nada, **no dispara**.

4. **Ráfaga:** 20 disparos, uno cada 0,3 s (el enunciado pide 12; los 8 de más cubren impactos no registrados). Cada disparo se lanza en su instante exacto respecto al audio de 6,2 s, para que el sonido coincida.
5. **Giros grandes en tramos de 45°** porque el SDK solo garantiza unos 55° por orden.

4.4. Los problemas reales y cómo se resolvieron

Esta es la parte más rica para explicar con honestidad.

Problema	Causa	Solución
El robot detectaba al rival (0,96–1,00) pero no disparaba; imágenes con franjas o viejas	El SDK deja el vídeo en una cola de 64 imágenes que nadie vacía mientras se analiza: se llena y llegan imágenes antiguas o dañadas	Un hilo lee el vídeo siempre y guarda solo la última; cada análisis espera una imagen tomada con la torreta ya quieta
La torreta apuntaba donde *estaba* el robot, no donde está	La imagen llega con ≈0,25 s de retraso por wifi	Se guarda un historial de ángulos de torreta y se usa el ángulo **del instante de la foto** (`angulo_en`)
Rival en movimiento	El robot se mueve mientras se analiza la imagen	Seguimiento: velocidad angular prevista por geometría + corrección limitada al 80 % del error; se analiza la imagen más nítida (varianza del laplaciano) de los últimos 0,2 s
El robot se desviaba al ir recto	Las ruedas decían "recto" pero el chasis giraba	Bucle de control con odometría + **IMU**: rampa de frenada, corrección de deriva lateral y de rumbo
La IMU "giraba" parada (8° en 60 s)	Deriva del sensor	Si lleva >3 s quieto, el cambio se toma como deriva y se mueve la referencia
Corrección de rumbo en el sentido equivocado	Se supuso el signo	Ahora se **mide**: giro de prueba de 0,3 s y se mira hacia dónde cambia la IMU
`chassis.move` iba a ≈0,06 m/s	El SDK no convierte a m/s reales	Se usa `drive_speed`
Sillas cercanas detectadas como robot (umbral bajo)	Cajas enormes	Ninguna caja más alta que media imagen es un robot (un robot así estaría a <0,5 m)
Sonido de la SCAR apenas audible	Clip de 0,19 s, el altavoz corta el inicio, y se subía por FTP en cada disparo	+9 dB, 0,3 s de silencio al principio y una sola pista de 6,2 s

4.5. El robot camuflado (y la lección sobre aliados)

La idea inicial era un **señuelo**: algo con aspecto de robot al que *no* disparar. Se probó con un RoboMaster disfrazado de detective y cuatro robots "aliados" generados con IA y marcados en verde. **Resultado: el detector los reconoce a todos como robots** (confianzas de 92,7 a

99,9 %). Tapando partes del detective la confianza apenas baja; solo cae al 52 % si se tapan chasis, ruedas y cañón a la vez. Conclusión: el modelo ha aprendido el *concepto* de robot, no una pieza.

Así la idea se transformó: el disfrazado pasa a ser un objetivo más (el "robot camuflado" de P1) y la identificación de aliados queda como **trabajo futuro con una marca de color**, no una forma: los aliados tienen entre 5,7 y 12,5 % de verde intenso en su caja y el detective 0 %, así que una regla ">2 % de verde = aliado" los separa. No se implementó por riesgo de falsos positivos con la luz del laboratorio.

El detective impreso en papel A3 da 0,72–0,97 con la cámara del robot (casi siempre <0,95); por eso en la zona izquierda de P1 se usa un **umbral reducido a 0,7**, limitado a cajas de menos de media imagen y a ese rango de ángulos. En pantalla de tableta funciona mucho peor (el brillo hace que la cámara baje la exposición).

4.6. Resultados de la demo oficial (12/11, en clase)

Con el detector en GPU. Los 4 rivales eliminados, **20 disparos cada uno (80 en total), registrados por los rivales**: rival 1 (confianza 1,00), robot camuflado (0,99), rival 2 en movimiento (1,00) y rival 3 escondido en una caja y detrás del robot (1,00).

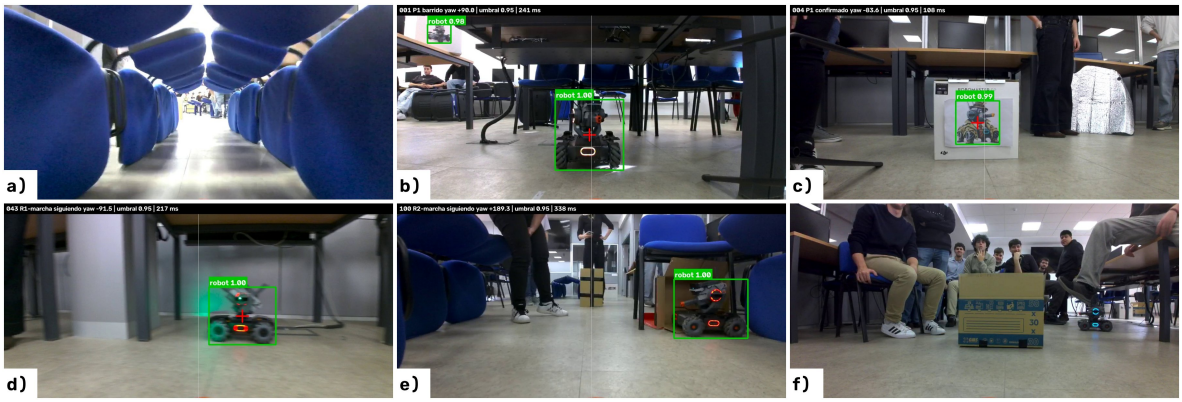


Figura 5: Cámara del robot en la demo oficial: túnel, rival 1, camuflado, rival 2, rival 3 y baile.

El intento sorpresa del profesor: puso un robot tapado con una tela (0,85–0,91) y la foto de un robot entre humo (0,59): **no disparó a ninguno** (bajo el umbral 0,95). En la recta final puso un robot con partes rojas, de un modelo que no está en nuestros datos: lo detectó (0,96–0,99) y le disparó. *Lectura honesta:* el detector generaliza a robots distintos, pero un robot tapado o una imagen borrosa no alcanzan 0,95; es el precio de un umbral alto para evitar falsos positivos.

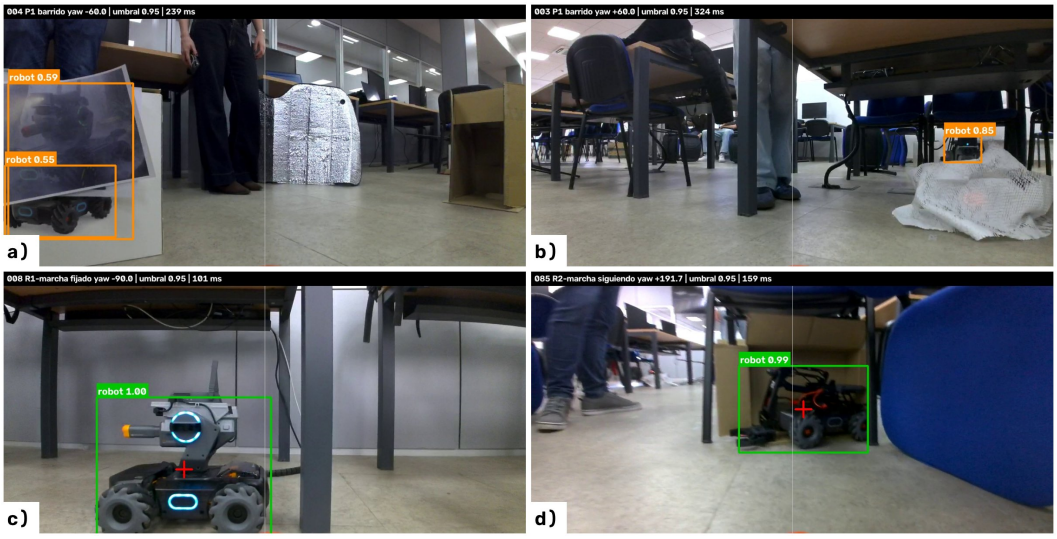


Figura 6: Intento sorpresa del profesor.

4.7. Seguridad (muy preguntable)

- Toda orden `drive_speed` caduca a los 0,5 s: si el programa se cuelga, el robot se para solo.
- `cerrar()` en un `finally`: con error o Ctrl+C se para el chasis, se centra la torreta y se apagan los LED.
- Batería: con menos del 10 % no arranca; con menos del 20 % avisa.
- Disparar solo a robots: umbral 0,95, filtro de cajas altas, distancia máxima en P1 y confirmación antes de la ráfaga.
- El rival móvil nunca pasa de 0,1 m/s y exige el número de serie del rival para no conectarse por error a nuestro robot.

4.8. Mejoras implementadas pero desactivadas

`SEGUIMIENTO`, `BARRIDO_CONTINUO`, `MEMORIA`, `VIGILANCIA` y `LEER_ANGULO_GIMBAL` están programadas (61 pruebas en simulación) pero **`a False`** porque no se validaron en el laboratorio: la demo hace lo probado y las mejoras se activan una a una. Es una decisión de ingeniería defendible (no incluir en la demo lo que no está comprobado) y conviene explicarla así.

5. La interfaz del mando

Interfaz web para manejar el robot desde el portátil, con teclado y ratón o mando de PS4. Muestra el vídeo con las detecciones, mueve chasis y torreta, dispara y lanza bailes con los *mismos topes de velocidad* que la demo. Probada con el robot real el 11/11: vídeo a 8–16 imágenes por segundo y detecciones con confianzas de 0,97 a 1,00.



Figura 7: Interfaz del mando con el robot real.

Decisiones de diseño interesantes:

- **Un solo hilo habla con el SDK** (`conductor.py`) para que dos órdenes no se pisen. El servidor y el detector solo dejan datos; el conductor decide. Bucle de control cada 25 ms.
- **Señal de vida:** el navegador manda el control cada 50 ms; si no llega nada en 0,3 s, el robot se para y se desarma (pestaña cerrada, wifi caído...).
- **Una sola sesión de control:** la última pestaña que se conecta se queda el control (la anterior se cierra con el código 4001) y, al cambiar de propietario, el robot se para. El servidor solo escucha en `127.0.0.1` y rechaza páginas de otro origen para que otra web no pueda mover el robot.
- **Vídeo MJPEG** sobre HTTP (`multipart/x-mixed-replace`), reducido a 960×540.
- **Disparo automático opcional:** solo si hay caja $\geq$0,95 bajo la mira, la detección tiene $<$0,5 s y la torreta no se ha movido más de 2° desde la foto.
- **Bailes como funciones del tiempo** (Moonwalk, Waka Waka, Super Estrella, Risa). Cada uno se *simula* antes de usarse: se integra el movimiento y se comprueba que no pasa de 0,5 m/s ni 30°/s, cabe en 1×1 m y **vuelve al punto de partida**.
- Los controles 3D con Three.js van copiados en local, porque en la wifi del robot no hay internet.

6. Material de apoyo para la presentación

- **Presentación** (`presentacion/`): 23 diapositivas en React + Vite. Teclas: flechas, `G` índice, `N` notas del presentador, `F` pantalla completa. Arranque: `npm install && npm run dev` (puerto 5190).

- **Web en GitHub Pages** (`web/`): figuras interactivas de los tres bloques; la del Bloque 3 muestra la demo foto a foto, qué vio el detector y cuánto duró cada fase.
- **Memoria**: incluye la tabla "Respuesta a la revisión anterior" (16 comentarios de la entrega 2, todos atendidos): SGD citado, esquema de bloques, una etapa frente a dos, batch size justificado, curva PR, FPN...

7. Mapa de decisiones: qué salió bien, qué falló y por qué

Los profesores insistieron en la **justificación**: no se valora tanto que algo funcione como saber *por qué* funcionó o falló y qué se aprendió. Esta sección reúne las decisiones del proyecto con ese formato. Cada fila se puede contar en 20 segundos.

7.1. Decisiones que salieron bien

Decisión	Por qué se tomó	Qué demostró
Entrenar con nuestros datos y evaluar con los de otros grupos	Los fotogramas de un vídeo se parecen entre sí (74 % de vecinos del mismo vídeo): mezclarlos inflaría el resultado	Test 0,9523 ≈ validación 0,9513: la medida es honesta, no hay fuga
Escribir la regla de selección antes de abrir el test	Evitar elegir modelo mirando el test	Nadie puede decir que el modelo se ajustó al test; empates <0,5 pp se resuelven por coste
Aumento propio en vez del del notebook	El del notebook recorta 640×480, ensancha los robots un 33 % y pierde el 14,1 % de ellos; el nuestro, 0,7 %	+8 pp de AP en *difficult* y 28 % menos de robots perdidos
Marcar *difficult* y evaluarlos aparte	Un AP global alto esconde los casos duros	Se vio que el 64 % de los fallos son *difficult* (el 16 % de los robots)
Umbral 0,95 fijado en validación	En la demo un falso disparo es peor que perder un objetivo	No disparó al robot tapado ni a la foto con humo del profesor
Detector en un proceso aparte con GPU	SDK atado a Python 3.8; PyTorch de 3.8 no soporta la RTX 5070	De 2,2 s a 0,12 s por imagen sin tocar el SDK, y con respaldo a CPU
Hilo que lee el vídeo y guarda solo la última imagen	La cola del SDK (64 imágenes) se llenaba con imágenes viejas	Se acabó el "detecta pero no dispara"
Usar el ángulo de la torreta *del instante de la foto*	La imagen llega con 0,25 s de retraso	La puntería deja de ir siempre por detrás del objetivo
Calcular velocidades sumando tiempos	Giro 3 s + ver 2–4 s + voz 1,4 s + ráfaga 6,2 s	El ataque en marcha cabe en el tramo (14,5 s a 0,1 m/s)
Medir el sentido de la corrección con la IMU	Se había supuesto el signo y estaba mal	Corrige hacia donde debe, no al revés
Mejoras desactivadas si no están validadas	Sin probar en laboratorio no se puede garantizar que no rompan la demo	La demo hace solo lo comprobado; se activan una a una

7.2. Lo que falló (o se descartó) y la lección

Qué pasó	**Por qué**	**Lección / qué se hizo**
Resolución 1024 px	AP pequeños sube de 0,12 a 0,19 pero con 134 FP más y +62 % de latencia (75 → 122 ms)	Una mejora no es uniforme: se mide en todas las métricas, no solo en la que sube. Se descartó
SSD y SSDlite muy por debajo (AP 0,88 y 0,64)	Reducen la imagen a 300–320 px: los robots lejanos quedan minúsculos	Velocidad no compensa si se pierden los objetivos difíciles (AP *difficult* 0,43 y 0,27)
Tasas 10^{-5}, 10^{-1} y $2 \cdot 10^{-3}$	10^{-5} casi no aprende, 10^{-1} da NaN y $2 \cdot 10^{-3}$ alcanza el máximo en la época 1–2 y luego baja más de 1 pp	Entre 10^{-4} y 10^{-2} solo cambia la velocidad; se usó 10^{-3}
Programador con paciencia de 5	El estancamiento ocurre en 2–3 épocas	Reconocido en la memoria: haría falta una prueba de rango de la tasa
30 épocas	En 6 de 7 modelos el mejor *checkpoint* está entre las épocas 3 y 7	No faltaban épocas: hay sobreajuste temprano. Se redujo a 12–15 y se guarda el mejor *checkpoint*
Lotes de 1–2 imágenes	Suben antes pero bajan en la tercera época	Con lote 8 la subida es más regular; es el mayor que cabe en 12 GB
Perspectiva y giros de 90° en el aumento	Con albumentations 1.4.18 la perspectiva desplaza mal las cajas (prueba automática); los giros de 90° ponen el suelo arriba	Se probó cada transformación antes de usarla; se descartaron
Etiquetas de otros grupos incompletas	Ej.: una imagen con 4 robots y 1 sola caja convierte 3 detecciones correctas en FP	No se corrigieron para conservar la comparación; se declara como limitación
Borde superior de las cajas	Los cinco discrepamos (mediana de 7 px frente a 4–5 px) por gimbal y antenas	IoU entre anotadores 0,81, no mayor. Cerrar el criterio queda pendiente (T01)
Robots “aliados” (señuelo)	El detector los reconoce a todos como robot (92,7–99,9 %); tapando partes apenas baja	El modelo aprendió el concepto, no una pieza. Idea reconvertida: robot camuflado = objetivo más; aliados con marca de color, como trabajo futuro
Robot sin disparar pese a detectarlo (0,96–1,00)	Imágenes con franjas o antiguas por la cola de vídeo	Primer gran bug de integración; se resolvió con el hilo de última imagen
Avance a 0,06 m/s y desvío al ir recto	`chassis.move` no da m/s reales; las ruedas decían “recto” pero el chasis giraba	`drive_speed` más bucle odometría + IMU
Sillas y robots del fondo como robot	Con umbral bajo, cajas enormes (robot a <0,5 m)	Filtro de caja alta y distancia máxima
Tramos del recorrido del plano	Las pruebas de laboratorio obligaron a corregir tramos (+15 % tras el túnel)	Medir en el laboratorio, no confiar en el plano

Qué pasó	Por qué	Lección / qué se hizo
Robot tapado (0,85–0,91) y foto con humo (0,59) sin disparo	Quedan bajo el umbral 0,95	Es el *coste* de un umbral alto: se asume y se explica

Idea clave

Patrón que se repite y que conviene verbalizar: **casi todos los fallos vinieron de suponer algo en lugar de medirlo** (el signo de la IMU, los metros del plano, la cola de vídeo, la latencia de la cámara, la resolución como mejora). Las decisiones que funcionaron se tomaron con una medida o una regla fijada antes.

8. Preguntas probables y cómo responderlas

Pregunta probable: ¿Por qué dos conjuntos distintos para entrenar y evaluar?

Entrenamos con nuestros 10 vídeos y validamos/testeamos con los conjuntos de otros grupos de la asignatura. Así no hay fuga entre conjuntos (los fotogramas de un mismo vídeo se parecen mucho: el vecino más parecido es del mismo vídeo en el 74 % de los casos) y medimos en escenas que el modelo no ha visto. Coste: no controlamos sus etiquetas.

Pregunta probable: ¿Qué aportó más, el aumento o la arquitectura?

Con el aumento fijado, la arquitectura influye más que la tasa o el perfil de aumento. Pero el aumento fue la mejora más importante *para los* difficult: +8 pp de AP@0,5 en ellos y 28 % menos de robots perdidos.

Pregunta probable: ¿Por qué AP@0,5 alto pero AP 0,50:0,95 bajo?

El modelo encuentra los robots pero las cajas no se ajustan tanto. En parte es por las etiquetas de otros grupos (criterio distinto con antenas y robots cortados): ni siquiera nosotros coincidíamos en el borde superior.

Pregunta probable: ¿Cómo sabéis que no habéis sobreajustado al test?

La regla de selección se escribió antes de abrir el test y cada modelo se evaluó una única vez, con el umbral ya fijado en validación. El test no se usó para elegir nada.

Pregunta probable: ¿La demo es robusta o funcionó una vez?

Hubo pruebas de laboratorio del 4 al 11/11 que corrigieron cola de vídeo, tramos de recorrido (+15 % tras el túnel), sillas y robots del fondo. La demo oficial del 12/11 salió completa y el intento sorpresa del profesor mostró los límites (robot tapado y foto con humo no alcanzan 0,95). Es una demo *con límites conocidos y explicados*.

Pregunta probable: ¿Por qué no usasteis la GPU desde el principio?

Porque el SDK exige Python 3.8, cuyo PyTorch no admite la RTX 5070. La solución fue un proceso aparte con Python 3.11/CUDA 12.8 y comunicación local, con respaldo a CPU.

Pregunta probable: ¿Qué haríais con más tiempo?

(1) Mejorar los *difficult* (0,57–0,69 en test) con datos reales de robots tapados y lejanos; (2) varias semillas para cerrar diferencias <0,5 pp; (3) distinguir aliados con una marca de color; (4) activar y validar el seguimiento continuo y la vigilancia del recorrido; (5) incluir secuencias del circuito real y con poca luz en el entrenamiento; (6) medir el retardo del vídeo (es el único `[PENDIENTE]` técnico que queda en la memoria).

Pregunta probable: ¿Qué fue lo que peor salió y qué aprendisteis?

Lo más costoso fue la integración, no la red: la cola de vídeo que daba imágenes viejas (el robot detectaba y no disparaba), el retraso de 0,25 s de la cámara y la GPU incompatible con el SDK. Aprendimos que hay que *medir* en lugar de suponer, y que separar procesos y tener un respaldo (CPU) convierte un problema bloqueante en uno controlado.

Pregunta probable: ¿Por qué falló la idea de los aliados?

Porque el detector aprendió el concepto de robot, no una pieza concreta: reconoce incluso robots disfrazados o generados con IA (92,7–99,9 %). Distinguirlos exige una señal distinta de la forma, como una marca de color (5,7–12,5 % de verde frente a 0 %); no se implementó por riesgo de falsos positivos con la luz del laboratorio.

9. Cifras para tener en la cabeza

Vídeos / duración / fotogramas	10 / 643,8 s / 19 508
Imágenes extraídas / tras depurar	808 / 796
Robots / *difficult* (originales)	1420 / 375
Entrenamiento tras aumento	3184 imágenes
Validación / test (asignatura)	672 / 718 imágenes
IoU medio entre anotadores	0,81
Modelo final	Faster R-CNN v2, lr 10^{-3}, época 3
AP@0,5 test (todos / *difficult*)	0,9523 / 0,6921
Umbral / TP / FP / FN en test	0,95 / 1606 / 94 / 115
Latencia modelo final / MobileNet	96 ms / 16 ms
Latencia en demo: CPU / GPU	2,2–2,4 s / ≈0,12 s
Disparos por robot / exigidos	20 / 12
Velocidades máximas	0,5 m/s y 30°/s
Entrenamientos en la cadena	13 (7 arquitecturas + variantes de tasa)